

CS-200 — Partie 2c : I/O, interruptions, DMA et timer

Oct 9, 2026 · @Gab

Ce cours répond à une question : comment le processeur communique-t-il efficacement avec des périphériques plus lents et imprévisibles que lui ? Il présente trois mécanismes, chacun corrigeant une faiblesse du précédent : le polling, les interruptions et le DMA. Le timer vient en dernier, comme périphérique qui génère des interruptions périodiques.

1. Le polling (slide 2)

Le CPU lit périodiquement un registre de statut de chaque périphérique (I/O mappée en mémoire, un simple `lw`) pour savoir s'il a quelque chose à signaler.

Ce mécanisme impose un compromis. Soit T la période entre deux vérifications et c le coût en cycles d'une vérification. Le délai de réaction est au pire T , et la fraction du CPU consommée vaut $c \times (\text{vérifications par seconde}) / f_{\text{clk}}$. Réagir vite exige un T petit, donc un coût élevé. De plus, la plupart des vérifications ne trouvent rien : ce travail est perdu.

Exemple classique (Patterson & Hennessy), avec un CPU à 500 MHz et un polling qui coûte 400 cycles :

Périphérique	Vérifications par seconde	Cycles par seconde	Part du CPU
Souris	30	12 000	$\approx 0,002 \%$
Disque à 4 MB/s, blocs de 16 octets	250 000	10^8	20 %

Les 20 % du disque sont consommés même quand il ne transfère rien. Le polling devient prohibitif dès qu'un périphérique est rapide ou exige une réaction immédiate.

2. L'idée de l'interruption (slides 3–5)

On inverse la responsabilité : le périphérique demande l'attention au lieu d'attendre qu'on vienne le voir.

Sur la slide 3, le programme principal compte les bits à 1 d'un mot (popcount). Il boucle 32 fois : `and` avec 1, accumulation dans `t1`, décalage à droite. Entre deux instructions (ici après le `add`), le contrôle saute de manière asynchrone vers `read_adc`. Cette routine lit

la sortie du convertisseur analogique-numérique à l'adresse `0xffffffff0 + 8`, la range dans `BUFFER`, puis rend la main. Le panneau « GIVE WAY » résume l'idée : le programme cède le passage.

La slide simplifie deux points qui rendent le mécanisme délicat :

- **read_adc écrase `t0`, `t1` et `t2`**, exactement les registres de la boucle. Au retour, `t0` (le mot en cours de décalage) et `t1` (le compteur) seraient corrompus. La convention d'appel ne s'applique pas ici : avec un `jal`, l'appelant sait qu'il appelle et sauvegarde ses `t*`, alors que le programme interrompu ne sait rien. Le handler doit sauvegarder et restaurer tous les registres qu'il touche, temporaires compris.
- **Le `ret` ne peut pas fonctionner tel quel.** `ret` signifie `jalr x0, 0(ra)`, mais aucun `jal` n'a placé l'adresse de retour dans `ra`, et il ne faut pas qu'un mécanisme le fasse, sinon le `ra` du programme principal serait détruit. Le matériel sauvegarde donc le PC dans un registre dédié, et le retour passe par une instruction spéciale. En RISC-V : `mepc` et `mret`, vraisemblablement vus plus loin dans le cours.

La slide 4 fait le lien avec les callbacks, listeners, signaux Unix et promises. La différence est fondamentale : dans ces mécanismes logiciels, un runtime, un compilateur ou un OS décide à quel moment le callback s'exécute, souvent à un point bien défini comme une boucle d'événements. Ici, c'est le matériel qui détourne l'exécution, potentiellement entre n'importe quelles deux instructions.

La slide 5 montre la vue matérielle « très naïve ». Le PC est normalement mis à jour par l'additionneur (+1 sur la slide, donc un adressage par mot ; +4 en RISC-V). Un multiplexeur commandé par la ligne IRQ permet de charger à la place l'adresse du handler. Il manque au minimum la sauvegarde du PC courant.

Lien avec le processeur multicycle de la partie 2a : la vérification de l'IRQ se place naturellement dans la FSM, avant l'état Fetch1. Si une interruption est en attente, on part vers des états qui sauvegardent le PC et chargent l'adresse du handler. L'instruction courante est ainsi terminée et la suivante n'a pas commencé.

3. Les questions à résoudre (slide 6)

Trois problèmes, traités dans les slides suivantes :

1. **Qui a demandé ?** Soit le logiciel interroge tous les périphériques après l'interruption (du polling, mais seulement quand on sait qu'il y a quelque chose), soit le périphérique s'identifie lui-même : c'est une interruption vectorisée.
2. **Les priorités.** Un contrôleur réseau dont le tampon déborde ne peut pas attendre ; un clavier, si.

3. **L'impact sur l'exécution en cours.** Dans un processeur multicycle, on finit l'instruction en cours. Dans un processeur pipeliné, plusieurs instructions sont en vol. Il faut alors une interruption précise : toutes les instructions avant un certain point ont modifié l'état, aucune après.

4. Partager une seule ligne IRQ : le wired-OR (slide 7)

On veut $IRQ = IRQ_0 \text{ OR } IRQ_1 \text{ OR } \dots \text{ OR } IRQ_n$ sans porte OR à n entrées, car les périphériques sont répartis sur un bus et on veut pouvoir en ajouter sans refaire le circuit.

La solution repose sur des sorties à trois états :

- Un périphérique qui demande l'attention **force** la ligne à 1 (pilotage fort).
- Un périphérique qui ne demande rien se met en **haute impédance (Z)** : il est électriquement déconnecté.
- Une **grande résistance** relie la ligne à 0, une « voix faible » qui gagne seulement si personne ne pilote la ligne.

Si au moins un périphérique force 1, la ligne vaut 1. Si tous sont en Z, la résistance tire la ligne à 0. On obtient un OR décentralisé.

Des sorties normales ne conviennent pas : si un périphérique pilote 0 et un autre 1 sur le même fil, on crée un court-circuit entre l'alimentation et la masse. Le Z est indispensable.

La slide précise « not quite the real circuit ». En pratique, on utilise souvent des sorties open-drain (ou open-collector) qui ne savent que tirer vers 0, avec une résistance de pull-up et un signal actif bas (/IRQ). C'est le même principe, inversé, et la même notation que /DTAC dans la partie 2b.